

layout_stride static extents default constructor fix

Document #: P2763
Date: 2023-01-13
Project: Programming Language C++
LWG
Reply-to: Christian Trott
<crtrott@sandia.gov>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Mark Hoemmen
<mhoemmen@nvidia.com>
Nevin Liber
<nliber@anl.gov>

Contents

- [1 Revision History](#)
 - [1.1 Initial Version 2023-01 Mailing](#)
- [2 Rationale](#)
- [3 Possible Fixes](#)
 - [3.1 Delete default constructor for fully static extents](#)
 - [3.2 Construct a layout_right equivalent mapping for fully static extents](#)
 - [3.3 Preference for Resolution](#)
 - [3.4 Proposed Wording](#)

§ 1 Revision History

§ 1.1 Initial Version 2023-01 Mailing

§ 2 Rationale

During work on the padded layouts project an oversight in the `layout_stride` definition was discovered for a corner case. Specifically, the default constructor of a `layout_stride` with fully static extents will produce an invalid mapping.

The default constructor is defaulted, which means that the array for the `strides` member will be all zeros. As such the `mappings operator()` will return always zero for a defaulted `layout_stride::mapping`. In the case of at least one dynamic extent this is not an issue, because the valid index space for calling `operator()` is empty. Thus the statement that `layout_stride::mapping` is always unique is not violated. However for fully static (non zero) extents, the valid index space is not empty, and thus no precondition is violated if a user calls `operator()` with a valid multidimensional index.

```
layout_stride::mapping<extents<int, 4>> map;  
// map.is_unique() == true;  
// map(0) == 0;  
// map(1) == 0;  
// map(2) == 0;  
// map(3) == 0;
```

Note: this issue does NOT affect mappings with extents of rank zero. It also does not affect `layout_left` and `layout_right` mappings. Their default constructed mappings with static extents are fully useable.

```
layout_left::mapping<extents<int, 4>> map;  
// map.is_unique() == true;  
// map(0) == 0;  
// map(1) == 1;  
// map(2) == 2;  
// map(3) == 3;
```

We believe that this issue is a defect in the C++23 draft and should be considered for defect resolution process.

§ 3 Possible Fixes

§ 3.1 Delete default constructor for fully static extents

One option is to simply make the default constructor unavailable for `layout_stride` mappings of fully static extents.

The wording change would simply add a `requires` clause to the class synopsis

In subsection 24.7.3.4.7.1 [mdspan.layout.stride.overview] replace:

```
// [mdspan.layout.stride.cons], constructors  
constexpr mapping() noexcept = default;  
constexpr mapping(const mapping&) noexcept = default;
```

With:

```
// [mdspan.layout.stride.cons], constructors  
constexpr mapping()  
    requires(extents_type::rank_dynamic() > 0 || extents_type::rank() == 0)  
    noexcept = default;  
constexpr mapping(const mapping&) noexcept = default;
```

Note that this approach mirrors the approach of `mdspan` itself, where the default constructor requires `rank_dynamic()>0`. For `mdspan` we can't avoid that, since there is no way to generate a valid `data_handle`.

§ 3.2 Construct a `layout_right` equivalent mapping for fully static extents

If one would want to preserve the default constructibility of every `layout_stride` mapping, one could create some valid strides in the case where all extents are static.

Specifically, one could constructs strides such that the resulting mapping is the standard C-Layout. Conveniently, we can obtain such strides from a default constructed `layout_right` mapping with the same extents.

It would be desirable to preserve the trivial constructibility of `layout_stride` mappings for all the cases where that is still possible.

In subsection 24.7.3.4.7.1 [`mdspan.layout.stride.overview`] replace:

```
// [mdspan.layout.stride.cons], constructors
constexpr mapping() noexcept = default;
constexpr mapping(const mapping&) noexcept = default;
```

With:

```
// [mdspan.layout.stride.cons], constructors
constexpr mapping()
    requires(extents_type::rank_dynamic()>0 || extents_type::rank()==0)
    noexcept = default;
constexpr mapping()
    requires(extents_type::rank_dynamic()==0 && extents_type::rank()>0)
    noexcept;
constexpr mapping(const mapping&) noexcept = default;
```

Add at the beginning of subsection 24.7.3.4.7.3 [`mdspan.layout.stride.cons`] insert:

```
constexpr mapping()
    requires(extents_type::rank_dynamic()==0 && extents_type::rank()>0)
    noexcept;
```

Effects: Direct-non-list-initializes `extents_` with `extents_type()`, and for all d in the range $[0, rank_)$, direct-non-list-initializes `strides_[d]` with `layout_right::mapping<extents_type>().stride(d)`.

§ 3.3 Preference for Resolution

We believe that it is preferable to preserve default constructibility of `layout_stride` for all specializations of `layout_stride` to simplify a number of generic programming cases – specifically, if layout policies are used directly in higher-level data structures for which the user wants to enable default constructibility.

§ 3.4 Proposed Wording

In subsection 24.7.3.4.7.1 [mdspan.layout.stride.overview] replace:

```
// [mdspan.layout.stride.cons], constructors
constexpr mapping() noexcept = default;
constexpr mapping(const mapping&) noexcept = default;
```

With:

```
// [mdspan.layout.stride.cons], constructors
constexpr mapping()
    requires(extents_type::rank_dynamic() > 0 || extents_type::rank() == 0)
    noexcept = default;
constexpr mapping()
    requires(extents_type::rank_dynamic() == 0 && extents_type::rank() > 0)
    noexcept;
constexpr mapping(const mapping&) noexcept = default;
```

Add at the beginning of subsection 24.7.3.4.7.3 [mdspan.layout.stride.cons] insert:

```
constexpr mapping()
    requires(extents_type::rank_dynamic() == 0 && extents_type::rank() > 0)
    noexcept;
```

Effects: Direct-non-list-initializes *extents_* with *extents_type()* , and for all *d* in the range $[0, rank_)$, direct-non-list-initializes *strides_[d]* with *layout_right::mapping<extents_type>().stride(d)*.